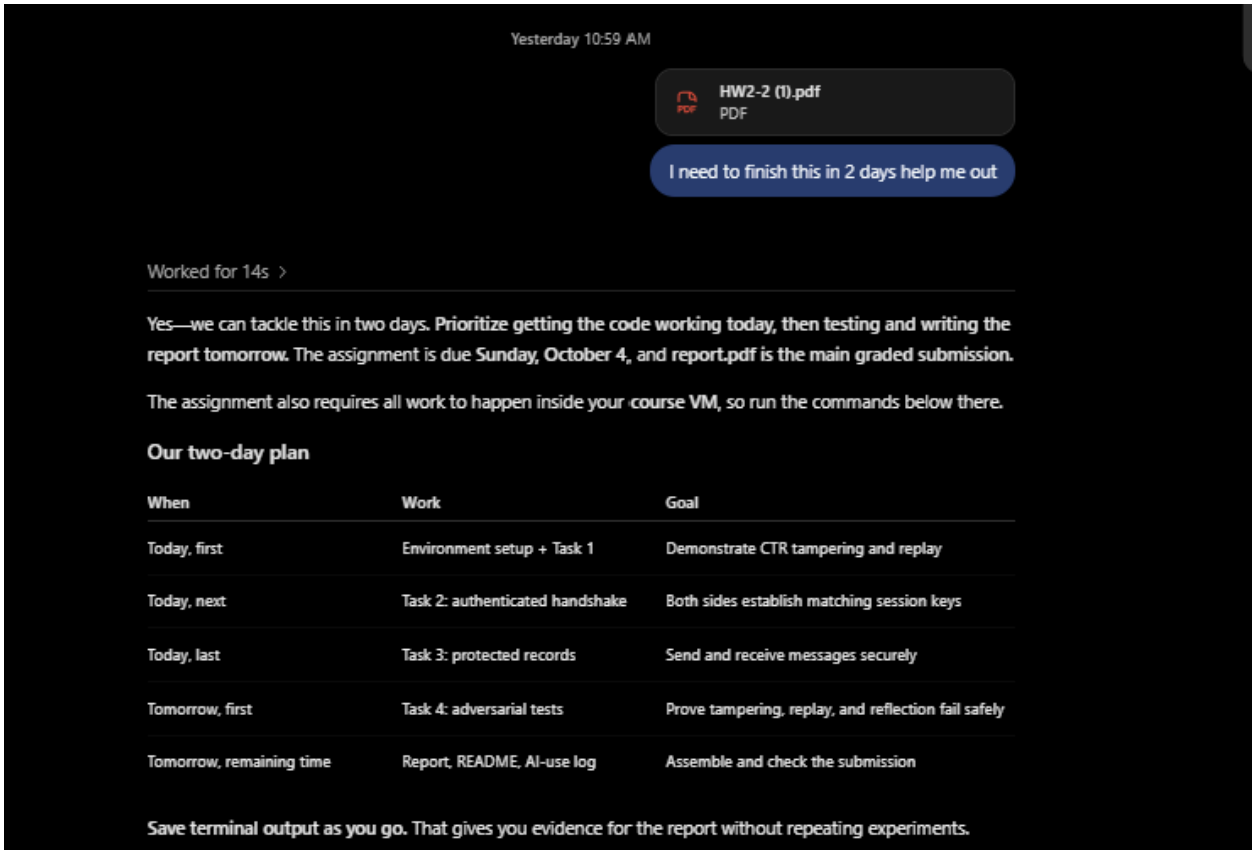
Ai chat logs



Yesterday 10:59 AM

HW2-2 (1).pdf
PDF

I need to finish this in 2 days help me out

Worked for 14s >

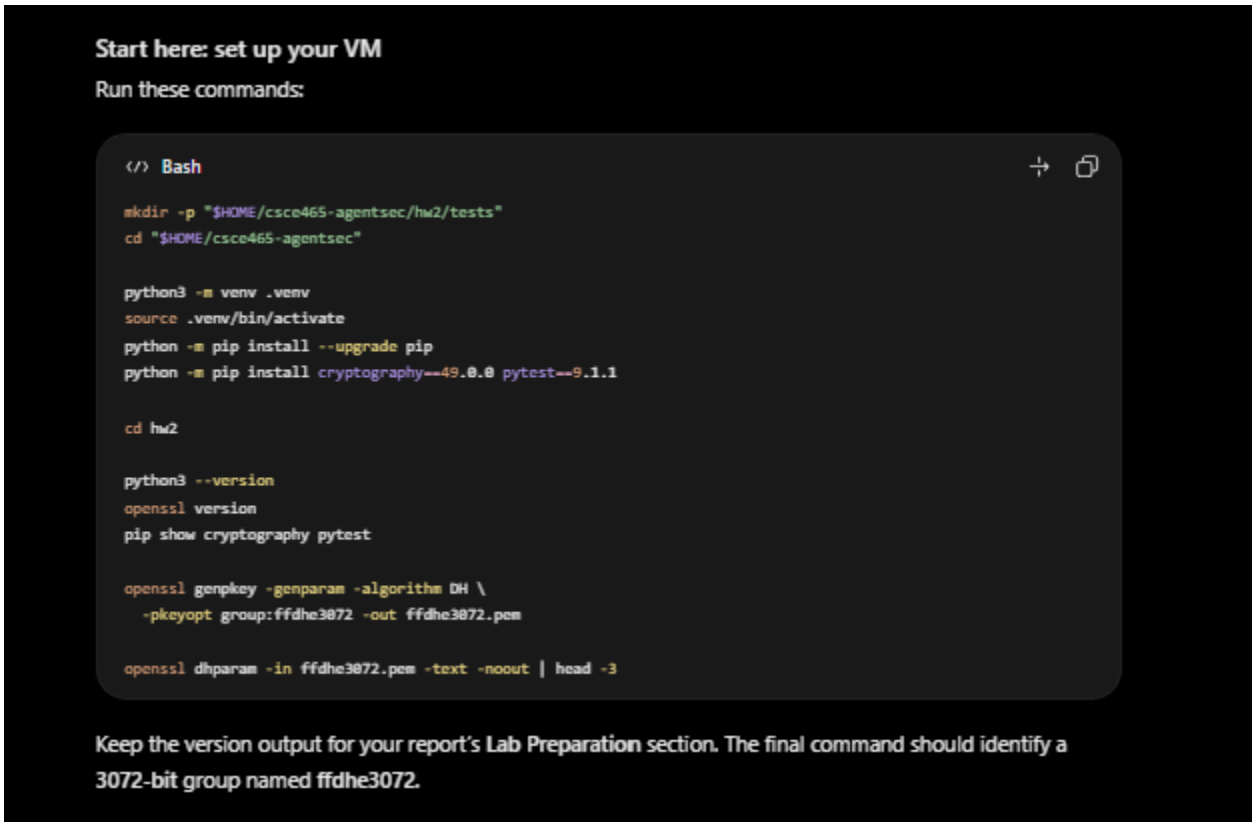
Yes—we can tackle this in two days. **Prioritize getting the code working today, then testing and writing the report tomorrow.** The assignment is due **Sunday, October 4,** and **report.pdf is the main graded submission.**

The assignment also requires all work to happen inside your **course VM**, so run the commands below there.

Our two-day plan

When	Work	Goal
Today, first	Environment setup + Task 1	Demonstrate CTR tampering and replay
Today, next	Task 2: authenticated handshake	Both sides establish matching session keys
Today, last	Task 3: protected records	Send and receive messages securely
Tomorrow, first	Task 4: adversarial tests	Prove tampering, replay, and reflection fail safely
Tomorrow, remaining time	Report, README, AI-use log	Assemble and check the submission

Save terminal output as you go. That gives you evidence for the report without repeating experiments.



Start here: set up your VM

Run these commands:

Bash

```
mkdir -p "$HOME/csce465-agentsec/hw2/tests"
cd "$HOME/csce465-agentsec"

python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install cryptography==49.0.0 pytest==9.1.1

cd hw2

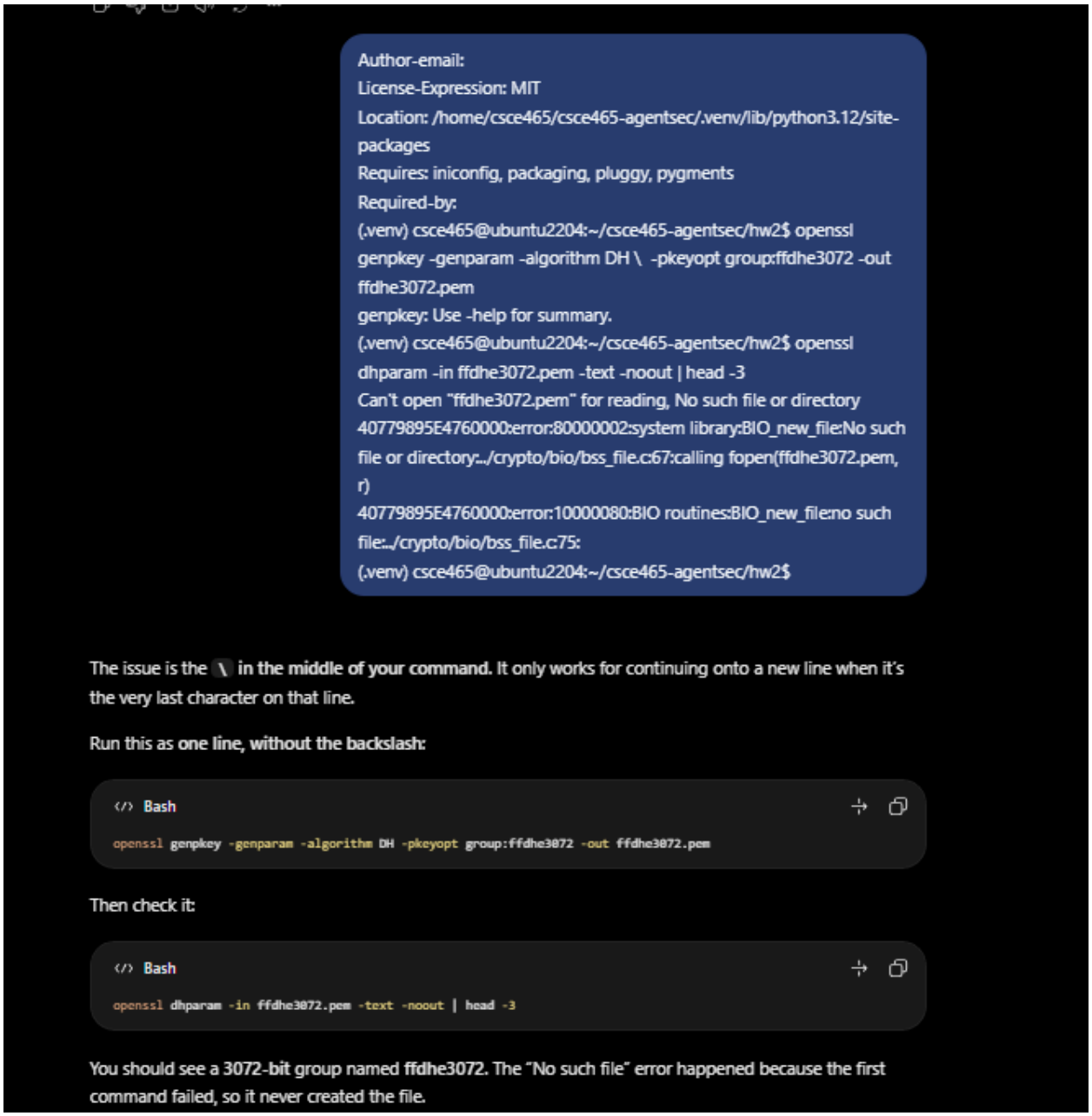
python3 --version
openssl version
pip show cryptography pytest

openssl genpkey -genparam -algorithm DH \
  -pkeyopt group:ffdhe3072 -out ffdhe3072.pem

openssl dhparam -in ffdhe3072.pem -text -noout | head -3
```

Keep the version output for your report's **Lab Preparation** section. The final command should identify a 3072-bit group named **ffdhe3072.**

Ai chat logs



Author-email:
License-Expression: MIT
Location: /home/csce465/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
(.venv) csce465@ubuntu2204:~/csce465-agentsec/hw2$ openssl genpkey -genparam -algorithm DH \ -pkeyopt group:ffdhe3072 -out ffdhe3072.pem
genpkey: Use -help for summary.
(.venv) csce465@ubuntu2204:~/csce465-agentsec/hw2$ openssl dhparam -in ffdhe3072.pem -text -noout | head -3
Can't open "ffdhe3072.pem" for reading, No such file or directory
40779895E4760000:error:80000002:system library:BIO_new_file:No such file or directory:../crypto/bio/bss_file.c67:calling fopen(ffdhe3072.pem, r)
40779895E4760000:error:10000080:BIO routines:BIO_new_file:no such file:../crypto/bio/bss_file.c75:
(.venv) csce465@ubuntu2204:~/csce465-agentsec/hw2$

The issue is the `\` **in the middle of your command.** It only works for continuing onto a new line when it's the very last character on that line.

Run this as **one line, without the backslash:**

Bash

```
openssl genpkey -genparam -algorithm DH -pkeyopt group:ffdhe3072 -out ffdhe3072.pem
```

Then check it:

Bash

```
openssl dhparam -in ffdhe3072.pem -text -noout | head -3
```

You should see a **3072-bit** group named **ffdhe3072.** The "No such file" error happened because the first command failed, so it never created the file.

Ai chat logs

I know that baseline.py is encryption without protection and tampering but how can I get started on creating it can you give me an outline

Start by splitting `baseline_ctr.py` into **three functions:** encrypt/decrypt, attacker relay, and the demonstration.

1. Import what you need

Python Run

```python
import os
import json
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
```

2. Write the CTR function

Inputs: **key, IV, and data.** Output: the transformed bytes.

Python Run

```python
def ctr(key, iv, data):
    # Create an AES cipher using CTR mode.
    # Create an encryptor.
    # Return context.update(data) + context.finalize().
    pass
```

With CTR, the same transformation can encrypt and decrypt because both operations XOR the data with the same keystream.

3. Write the attacker's relay

Give this function only the ciphertext—not the key.

Python Run

```python
def relay(ciphertext):
    original = b"READ"
    replacement = b"LIST"
    offset = len(b'{"action":"')

    # Copy ciphertext into a bytearray so you can change bytes.
    # Calculate original[i] XOR replacement[i] for each of the 4 bytes.
    # XOR each difference into ciphertext[offset + i].
    # Print the original bytes, replacement bytes, and XOR difference.
    # Return the modified ciphertext as bytes.
    pass
```

The attacker knows where `READ` appears because the command's format is fixed.

4. Set up the demonstration

Python Run

```python
def main():
    key = os.urandom(32)   # 32 bytes = 256-bit AES key
    iv = os.urandom(16)    # CTR starting counter
    plaintext = b'{"action":"READ","path":"notes.txt"}'

    # Encrypt plaintext using ctr().

    # Decrypt the original ciphertext and print the command.

    # Pass ciphertext through relay().
    # Decrypt the modified ciphertext and print the command.

    # Send that same modified ciphertext again.
    # Decrypt and print it again to demonstrate replay.
```

For "processing," use `json.loads()` and print the resulting dictionary. You don't need to execute a real command.

5. Call `main()`

Ai chat logs

No I dont have handshake.py but I know that handshake.py establishes shared keys between the gateway and node using Diffie-Hellman but can you help me make the code for the file I'm a little lost on where to start

Got it—let's build `handshake.py` from the beginning, **one working piece at a time.**

First, make both parties calculate the same DH secret. We'll add authentication afterward.

1. Open the file

From your `hw2` directory:

```bash
nano handshake.py
```

2. Add this starting code

```python
from pathlib import Path
from cryptography.hazmat.primitives import serialization


def load_group():
    # Load the standard DH parameters you generated with OpenSSL.
    path = Path(__file__).with_name("ffdhe3072.pem")
    return serialization.load_pem_parameters(path.read_bytes())


def main():
    parameters = load_group()

    # Each party generates its own fresh DH private key.
    gateway_private = parameters.generate_private_key()
    node_private = parameters.generate_private_key()

    # These public keys can be exchanged.
    gateway_public = gateway_private.public_key()
    node_public = node_private.public_key()
```

Save with **Ctrl+O → Enter → Ctrl+X**, then run:

```bash
python handshake.py
```

What each part means

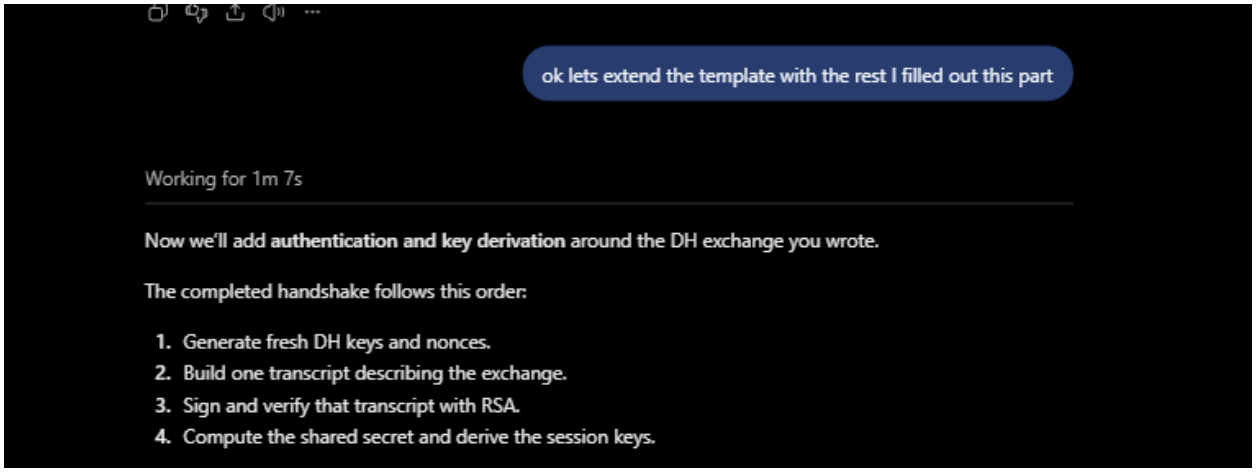
Code	Purpose
`load_group()`	Loads the shared public DH parameters
`generate_private_key()`	Creates each party's secret private value
`.public_key()`	Gets the public value that the other party receives
`.exchange(peer_public)`	Computes the shared secret
`assert`	Checks that both calculations match

This is only the first part of the assignment. It establishes a shared secret but doesn't verify who the other party is.

Once this runs, we'll extend the same file with:

1. Fresh nonces and the length-prefixed transcript.
2. RSA signing keys and signature verification.

Ai chat logs


ok lets extend the template with the rest I filled out this part
Working for 1m 7s
Now we'll add authentication and key derivation around the DH exchange you wrote.
The completed handshake follows this order:
1. Generate fresh DH keys and nonces.
2. Build one transcript describing the exchange.
3. Sign and verify that transcript with RSA.
4. Compute the shared secret and derive the session keys.